

PROMPT MESTRE — SENTINELCORE

Plataforma de Identidade, Segurança e Detecção de Ameaças

1. VISÃO GERAL DO PROJETO

O SentinelCore será desenvolvido como uma plataforma de segurança composta por dois repositórios independentes:

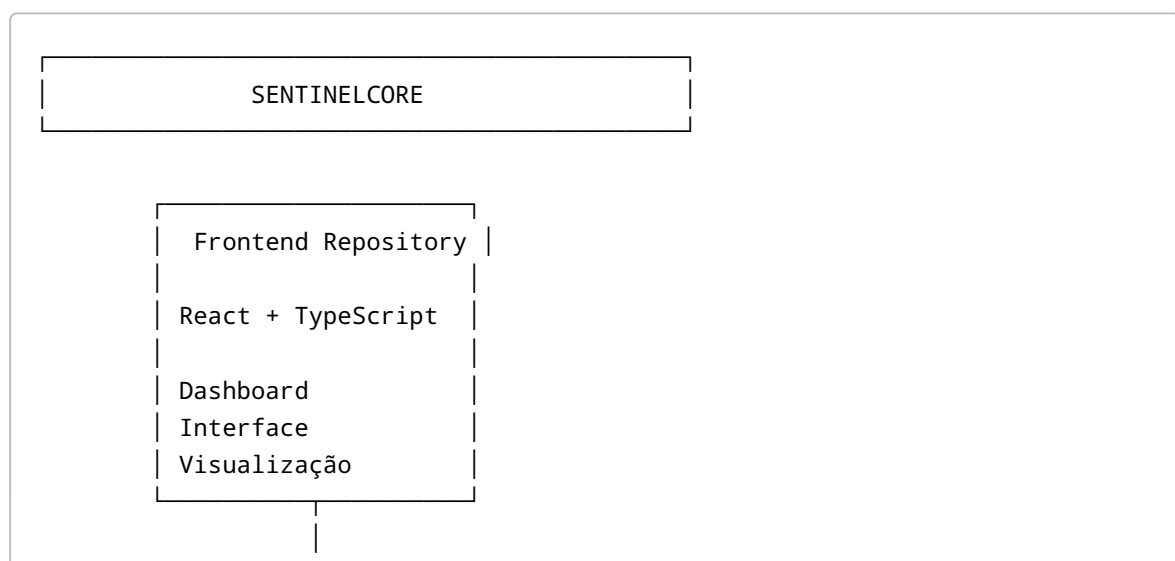
```
sentinelcore-backend/  
sentinelcore-frontend/
```

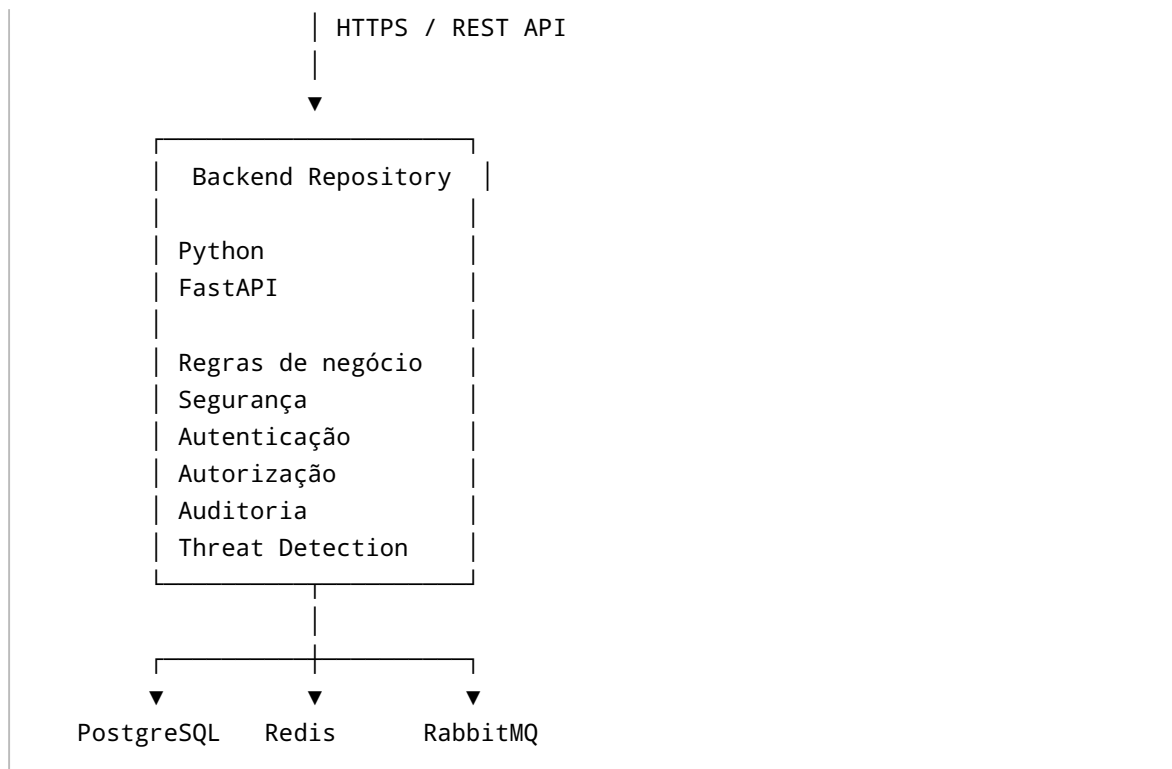
Cada repositório possui:

- seu próprio Git;
- seu próprio README;
- sua própria documentação;
- seu próprio ciclo de desenvolvimento;
- seus próprios testes;
- seus próprios commits;
- suas próprias responsabilidades.

A plataforma completa será formada pela integração entre os dois projetos.

2. ARQUITETURA GERAL





O frontend nunca deve possuir regras críticas de segurança.

O backend é a autoridade do sistema.

O frontend é um cliente do backend.

3. REPOSITÓRIOS

Repositório 1 — Backend

sentinelcore-backend/

Responsável por:

- domínio;
- regras de negócio;
- autenticação;
- autorização;
- usuários;
- sessões;
- auditoria;
- eventos;
- detecção de ameaças;
- análise de risco;
- incidentes;

- respostas automatizadas;
- API;
- persistência;
- mensageria;
- observabilidade.

Tecnologia principal:

Python

Repositório 2 — Frontend

sentinelcore-frontend/

Responsável por:

- interface;
- experiência do usuário;
- dashboard;
- visualização de dados;
- consumo da API;
- gerenciamento do estado da interface;
- navegação;
- formulários;
- feedback visual.

Tecnologias principais:

React
TypeScript

4. REGRA FUNDAMENTAL DE RESPONSABILIDADES

Backend

O backend decide:

O usuário pode fazer isso?

0 usuário está autenticado?

0 token é válido?

0 usuário possui permissão?

0 comportamento é suspeito?

Um incidente deve ser criado?

Frontend

O frontend decide:

Como essa informação será exibida?

Como o usuário interage com o sistema?

Qual tela será exibida?

Como apresentar um erro?

O frontend nunca deve ser considerado uma camada de segurança.

Esconder um botão não é autorização.

A autorização deve sempre acontecer no backend.

5. ESTRUTURA DO ECOSSISTEMA

A organização geral será:

```
projects/  
|  
├─ sentinelcore-backend/
```

```
| sentinelcore-frontend/
```

Os repositórios não devem compartilhar código diretamente.

A comunicação deve ocorrer através de contratos bem definidos.

```
Frontend
|
| HTTP
▼
Backend API
```

6. BACKEND — STACK

Linguagem

Python.

A versão deve ser definida no início do projeto e documentada.

Gerenciamento de ambiente

Utilizar exclusivamente:

uv

O `uv` será utilizado para:

- criação do ambiente;
- instalação de dependências;
- gerenciamento de dependências;
- execução do projeto;
- execução dos testes;
- execução das ferramentas;
- gerenciamento do lockfile.

Arquivos principais:

```
pyproject.toml
uv.lock
```

Não utilizar Poetry ou Pipenv como gerenciadores principais.

Framework

FastAPI.

Banco de dados

PostgreSQL.

Utilizar:

- SQLAlchemy Async;
 - asyncpg;
 - Alembic;
 - migrations;
 - constraints;
 - índices;
 - transações.
-

Cache

Redis.

Utilizar para:

- rate limiting;
 - dados temporários;
 - sessões quando necessário;
 - locks distribuídos quando necessário.
-

Mensageria

RabbitMQ.

Utilizar para:

- eventos de segurança;
 - processamento assíncrono;
 - comunicação entre componentes;
 - detecção de ameaças.
-

Infraestrutura

Docker e Docker Compose.

Observabilidade

Utilizar:

- logs estruturados;
- Prometheus;
- Grafana;
- OpenTelemetry.

7. FRONTEND — STACK

O frontend deve ser desenvolvido separadamente.

Tecnologias:

React
TypeScript

A ferramenta de build deve ser escolhida e documentada no início do projeto.

A escolha deve priorizar:

- simplicidade;
- velocidade;
- boa integração com TypeScript;
- experiência de desenvolvimento.

O frontend deverá possuir:

- gerenciamento de rotas;
- camada de comunicação com API;
- gerenciamento de autenticação;
- gerenciamento de estado;
- componentes reutilizáveis;
- tratamento de loading;
- tratamento de erro;
- validação de formulários.

8. REPOSITÓRIO DO BACKEND

Estrutura inicial:

```
sentinelcore-backend/
|
|— README.md
|— LICENSE
|— .gitignore
|— .editorconfig
|— .env.example
|
|— pyproject.toml
|— uv.lock
|
|— Dockerfile
|— docker-compose.yml
|
|— src/
|   └─ sentinelcore/
|       |
|       |— main.py
|       |
|       |— core/
|       |   └─ config/
|       |   └─ errors/
|       |   └─ logging/
|       |   └─ security/
|       |       └─ observability/
|       |
|       |— modules/
|       |   └─ identity/
|       |   └─ authentication/
|       |   └─ authorization/
|       |   └─ sessions/
|       |   └─ audit/
|       |   └─ security_events/
|       |   └─ threat_detection/
|       |   └─ risk_analysis/
|       |       └─ incidents/
|       |
|       └─ infrastructure/
|           └─ database/
|           └─ cache/
```

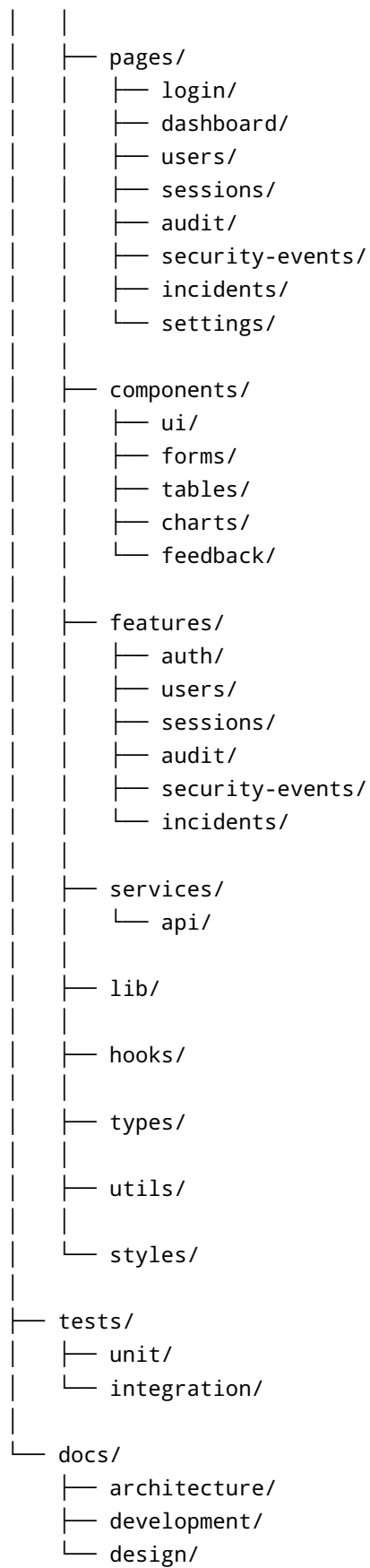


```
|
|
|       ├── messaging/
|       └── external_services/
|
|── tests/
|   ├── unit/
|   ├── integration/
|   └── e2e/
|
|── docs/
|   ├── architecture/
|   ├── security/
|   ├── development/
|   ├── operations/
|   └── decisions/
|
|── infrastructure/
|   ├── docker/
|   └── monitoring/
|
|── scripts/
```

9. REPOSITÓRIO DO FRONTEND

Estrutura inicial:

```
sentinelcore-frontend/
|
|── README.md
|── LICENSE
|── .gitignore
|── .editorconfig
|
|── package.json
|── package-lock.json
|
|── .env.example
|
|── public/
|
|── src/
|   |
|   ├── app/
|   |
|   ├── routes/
|   |
|   └── layouts/
```



Essa estrutura poderá evoluir.

Não criar diretórios apenas para tornar a estrutura visualmente complexa.

10. DOCUMENTAÇÃO CENTRAL

Como os repositórios são separados, cada um deve possuir sua própria documentação.

Porém, algumas decisões são compartilhadas.

A recomendação é possuir um terceiro repositório opcional:

```
sentinelcore-docs/
```

Esse repositório pode conter:

```
sentinelcore-docs/  
├── README.md  
├── architecture/  
│   ├── system-context.md  
│   ├── integration.md  
│   └── diagrams/  
├── product/  
│   ├── vision.md  
│   ├── problem-statement.md  
│   └── use-cases.md  
├── security/  
│   ├── threat-model.md  
│   └── security-principles.md  
└── roadmap/  
    └── roadmap.md
```

Caso não seja desejado um terceiro repositório, a documentação compartilhada deve ficar no backend.

Nesse caso:

```
sentinelcore-backend/docs/
```

será a fonte principal das decisões de arquitetura do sistema.

O frontend deverá referenciar essas decisões quando necessário.

11. CONTRATO ENTRE BACKEND E FRONTEND

A comunicação entre os repositórios deve ser tratada como um contrato.

O backend deve possuir uma API documentada.

A API deve definir:

- endpoints;
- métodos;
- parâmetros;
- request body;
- response body;
- códigos HTTP;
- erros;
- autenticação;
- autorização.

Exemplo:

```
POST /api/v1/auth/login
```

Request:

```
{
  "email": "user@example.com",
  "password": "password"
}
```

Response:

```
{
  "access_token": "...",
  "refresh_token": "...",
  "token_type": "bearer"
}
```

O frontend não deve presumir formatos.

O contrato deve ser documentado.

12. VERSIONAMENTO DA API

A API deve ser versionada:

```
/api/v1
```

Exemplo:

```
/api/v1/auth/login  
/api/v1/users  
/api/v1/sessions  
/api/v1/incidents
```

Alterações incompatíveis devem resultar em nova versão.

13. CONTRATOS E TIPOS

O backend deve definir schemas de request e response.

O frontend deve possuir tipos correspondentes.

Exemplo:

Backend:

```
class UserResponse(BaseModel):  
    id: UUID  
    email: str  
    status: str
```

Frontend:

```
interface User {  
    id: string;  
    email: string;  
    status: string;  
}
```

Esses contratos devem ser mantidos consistentes.

No futuro, pode ser avaliada a geração automática de tipos TypeScript a partir do OpenAPI.

Essa decisão deve ser registrada em ADR.

14. REGRA DE DESENVOLVIMENTO POR FASES

O backend e o frontend serão desenvolvidos em fases coordenadas.

Cada fase deverá especificar:

Backend:

- o que será criado;
- o que será alterado;
- quais endpoints existirão;
- quais eventos serão gerados;
- quais testes serão criados.

Frontend:

- quais telas existirão;
- quais componentes serão criados;
- quais endpoints serão consumidos;
- quais estados serão tratados;
- quais testes serão criados.

Uma funcionalidade completa deverá seguir:

```
Domínio
↓
Backend
↓
API
↓
Contrato
↓
Frontend
↓
Integração
↓
Testes
```

15. REGRA OBRIGATÓRIA DE COMMITS

Os repositórios possuem Git independente.

Portanto, cada repositório terá seus próprios commits.

Ao final de cada fase, deverão ser entregues:

Commit do backend

```
feat(auth): implement authentication flow
```

Commit do frontend

```
feat(auth): implement authentication interface
```

Não misturar alterações do backend e frontend em um único commit.

16. FLUXO DE CONCLUSÃO DE FASE

Ao finalizar uma fase:

Backend

Verificar:

- implementação;
- testes;
- documentação;
- migrations;
- endpoints;
- segurança.

Entregar:

```
git add .  
git commit -m "..."
```

Frontend

Verificar:

- interface;
- integração;
- estados;
- erros;
- loading;
- responsividade;
- testes.

Entregar:

```
git add .  
git commit -m "..."
```

Importante

Você não deve executar os commits.

Você deve apenas me fornecer os comandos.

Eu executarei os commits manualmente.

Depois que eu confirmar que os commits foram realizados, a próxima fase poderá começar.

17. FASE 0 — DEFINIÇÃO DO PRODUTO

Essa fase é compartilhada conceitualmente.

Definir:

- problema;
- público;
- personas;
- casos de uso;
- objetivos;
- glossário.

Criar no backend:

```
docs/product/
```

Ou no repositório central de documentação.

Não criar código ainda.

Commit

Backend:

```
docs(product): define SentinelCore product foundation
```

Frontend:

Se não houver alteração no frontend, não criar commit artificial.

18. FASE 1 — ARQUITETURA DO ECOSSISTEMA

Documentar:

```
Frontend
  ↓
API
  ↓
Backend
  ↓
Database / Cache / Messaging
```

Definir:

- fronteira dos repositórios;
- responsabilidades;
- comunicação;
- autenticação;
- contratos;
- versionamento da API.

Criar ADRs:

```
ADR-001: Separate backend and frontend repositories
ADR-002: Python backend
ADR-003: React frontend
ADR-004: Modular Monolith
ADR-005: API versioning
ADR-006: OpenAPI as API contract
```

Commit

Backend:

```
docs(architecture): define system boundaries and integration
```

Frontend:

```
docs(architecture): define frontend architecture
```

19. FASE 2 — BOOTSTRAP DO BACKEND

Criar:

- projeto uv;
- pyproject.toml;
- uv.lock;
- FastAPI;
- src layout;
- configuração;
- Docker;
- PostgreSQL;
- health check.

Executar:

```
uv sync
```

A aplicação deve subir.

Commit

```
feat(bootstrap): initialize backend foundation
```

20. FASE 3 — BOOTSTRAP DO FRONTEND

Criar:

- projeto React;
- TypeScript;
- estrutura de diretórios;
- configuração de ambiente;
- layout base;
- roteamento;
- página inicial.

O frontend deve executar independentemente do backend.

Commit

```
feat(bootstrap): initialize frontend foundation
```

21. FASE 4 — ARQUITETURA INTERNA DO BACKEND

Implementar:

- domain;
- application;
- ports;
- adapters;
- infrastructure;
- dependency injection;
- tratamento de erros.

A regra:

O domínio não deve depender do framework.

Commit

```
feat(architecture): establish backend application boundaries
```

22. FASE 5 — DESIGN SYSTEM DO FRONTEND

Criar:

- tokens;
- tipografia;
- espaçamento;
- cores;
- componentes base;
- botões;
- inputs;
- cards;
- tabelas;
- modais;
- feedbacks.

O frontend deve possuir consistência visual.

Commit

```
feat(ui): establish frontend design system
```

23. FASE 6 — IDENTIDADE

Backend

Implementar:

- usuário;
- credenciais;
- status;
- criação;
- ativação;
- desativação;
- hashing de senha.

Commit

```
feat(identity): implement user identity domain
```

Frontend

Implementar:

- tela de usuários;
- listagem;
- detalhes;
- estados;
- loading;
- erros.

Commit

```
feat(identity): implement user management interface
```

24. FASE 7 — AUTENTICAÇÃO

Backend

Implementar:

- login;
- access token;
- refresh token;
- logout;

- expiração;
- revogação;
- refresh token rotation.

Testar:

- token inválido;
- token expirado;
- replay;
- reutilização.

Commit

```
feat(auth): implement secure authentication flow
```

Frontend

Implementar:

- login;
- armazenamento seguro da sessão;
- proteção de rotas;
- logout;
- refresh de sessão;
- estados de carregamento;
- tratamento de expiração.

Nunca confiar apenas no frontend para proteger rotas.

Commit

```
feat(auth): implement authentication interface
```

25. FASE 8 — SESSÕES E DISPOSITIVOS

Backend

Implementar:

- sessões;
- dispositivos;
- IP;
- user-agent;
- criação;
- listagem;

- revogação.

Commit

```
feat(sessions): implement session management
```

Frontend

Criar:

- tela de sessões;
- dispositivos;
- localização aproximada quando disponível;
- data do último acesso;
- botão de revogação.

Commit

```
feat(sessions): implement session management interface
```

26. FASE 9 — AUTORIZAÇÃO

Backend

Implementar:

```
User
  ↓
Role
  ↓
Permission
```

Criar RBAC.

Posteriormente avaliar ABAC.

Commit

```
feat(authorization): implement role based access control
```

Frontend

O frontend deve:

- ocultar funcionalidades sem permissão para melhorar UX;
- apresentar mensagens adequadas;
- tratar 401;
- tratar 403.

Mas nunca considerar a interface como mecanismo de segurança.

Commit

```
feat(authorization): integrate permission-aware interface
```

27. FASE 10 — AUDITORIA

Backend

Registrar:

- login;
- falha de login;
- alteração de senha;
- mudança de permissões;
- criação de usuário;
- revogação de sessão;
- operações sensíveis.

Commit

```
feat(audit): implement security audit trail
```

Frontend

Criar:

- visualização de eventos;
- filtros;
- busca;
- paginação;
- detalhes do evento.

Commit

```
feat(audit): implement audit events interface
```

28. FASE 11 — EVENTOS DE SEGURANÇA

Backend

Criar eventos:

```
LOGIN_SUCCESS  
LOGIN_FAILED  
SESSION_CREATED  
SESSION_REVOKED  
PASSWORD_CHANGED  
PERMISSION_CHANGED
```

Definir:

- schema;
- metadata;
- correlation ID;
- timestamp.

Commit

```
feat(security-events): implement security event model
```

29. FASE 12 — RABBITMQ

Backend

Implementar:

```
Event  
↓  
Producer  
↓  
RabbitMQ
```


↓
Consumer

Adicionar:

- retry;
- dead-letter queue;
- idempotência.

Commit

```
feat(messaging): implement asynchronous security event processing
```

30. FASE 13 — THREAT MODELING

Documentar:

- ativos;
- ameaças;
- atores;
- vetores;
- impacto;
- mitigação;
- detecção;
- resposta.

Utilizar STRIDE.

Commit

```
docs(security): define threat model
```

31. FASE 14 — DETECÇÃO DE AMEAÇAS

Implementar:

- brute force detection;
- credential stuffing detection;
- suspicious login;
- account takeover patterns.

Criar Risk Score explicável.

Exemplo:

Novo dispositivo	+20
IP suspeito	+30
Falhas consecutivas	+25
Comportamento anormal	+25

Commit

```
feat(threat-detection): implement suspicious behavior detection
```

32. FASE 15 — INCIDENT MANAGEMENT

Backend

Implementar:

```
OPEN
INVESTIGATING
CONTAINED
RESOLVED
CLOSED
```

Criar:

- incidentes;
- evidências;
- timeline;
- severidade.

Commit

```
feat(incidents): implement incident management
```

Frontend

Criar:

- dashboard de incidentes;
- lista;
- detalhes;
- timeline;
- filtros;
- alteração de status;

- severidade.

Commit

```
feat(incidents): implement incident management interface
```

33. FASE 16 — RESPOSTA AUTOMATIZADA

Implementar:

- revogação de sessão;
- bloqueio de IP;
- exigência de MFA;
- bloqueio de conta;
- rate limiting adaptativo.

Toda ação deve ser:

- registrada;
- auditável;
- explicável.

Commit

```
feat(response): implement automated security response
```

34. FASE 17 — RATE LIMITING

Implementar:

- limite por IP;
- limite por usuário;
- limite por endpoint;
- Redis.

Avaliar:

```
Fixed Window  
Sliding Window  
Token Bucket
```

Escolher e documentar.

Commit

```
feat(security): implement distributed rate limiting
```

35. FASE 18 — OBSERVABILIDADE

Backend

Implementar:

- logs estruturados;
- métricas;
- correlation ID;
- tracing.

Métricas:

```
login_success_total  
login_failed_total  
security_events_total  
incidents_created_total  
request_latency
```

Commit

```
feat(observability): implement backend observability
```

Frontend

Implementar:

- estados de erro;
- monitoramento de falhas de API;
- feedback visual;
- tratamento de sessão expirada.

Commit

```
feat(observability): improve frontend error visibility
```

36. FASE 19 — SECURITY LAB

Criar simuladores controlados para:

- brute force;
- credential stuffing;
- login suspeito;
- comportamento anômalo.

Fluxo:

```
graph TD;
    SL[Security Lab] --> B[Backend];
    B --> SE[Security Event];
    SE --> TD[Threat Detection];
    TD --> I[Incident];
    I --> R[Response];
    R --> F[Frontend];
```

Os simuladores devem ser:

- locais;
- controlados;
- educacionais;
- defensivos.

Commit

Backend:

```
feat(security-lab): implement controlled security scenarios
```

Frontend:

```
feat(security-lab): visualize security scenarios
```

37. FASE 20 — CI/CD

Backend

Pipeline:

```
Push
↓
Lint
↓
Type Check
↓
Unit Tests
↓
Integration Tests
↓
Security Checks
↓
Build
```

Commit

```
ci(backend): implement continuous integration pipeline
```

Frontend

Pipeline:

```
Push
↓
Lint
↓
Type Check
↓
Unit Tests
↓
Build
```

Commit

```
ci(frontend): implement continuous integration pipeline
```

38. FASE 21 — DOCUMENTAÇÃO FINAL

Backend README

Deve explicar:

- o que é;
 - arquitetura;
 - stack;
 - como executar;
 - como testar;
 - variáveis de ambiente;
 - banco;
 - mensageria;
 - segurança;
 - documentação da API.
-

Frontend README

Deve explicar:

- o que é;
 - arquitetura;
 - stack;
 - como executar;
 - variáveis de ambiente;
 - como configurar o backend;
 - como executar testes;
 - estrutura de pastas.
-

39. REGRA DE INTEGRAÇÃO

Uma funcionalidade só é considerada completa quando:

```
Backend implementado
  ↓
Testes do backend
  ↓
API documentada
  ↓
Frontend integrado
  ↓
Estados tratados
  ↓
Testes do frontend
```

↓
Teste de integração

Nunca considerar uma funcionalidade pronta apenas porque:

O endpoint funciona

ou:

A tela aparece.

A funcionalidade precisa funcionar de ponta a ponta.

40. REGRA DE COMUNICAÇÃO DURANTE O DESENVOLVIMENTO

Para cada tarefa, analisar:

1. Problema

O que precisa ser resolvido?

2. Arquitetura

Qual módulo é responsável?

3. Backend

Quais entidades, casos de uso e endpoints serão necessários?

4. Contrato

Qual request e response existirão?

5. Frontend

Qual tela ou componente consumirá a funcionalidade?

6. Segurança

Existe algum risco?

7. Testes

Como validar?

8. Documentação

O que precisa ser atualizado?

9. Commits

Qual commit será criado em cada repositório?

41. REGRA DE REVISÃO

Ao final de cada fase, realizar uma revisão.

Verificar:

Código
Testes
Arquitetura
Segurança
Documentação
Integração

Depois entregar:

FASE CONCLUÍDA

BACKEND

Resumo:

...

Arquivos:

...

Testes:

...

Documentação:

...

Commit:

...

```
Comando:  
git add .  
git commit -m "..."
```

FRONTEND

Resumo:
...

Arquivos:
...

Testes:
...

Documentação:
...

Commit:
...

```
Comando:  
git add .  
git commit -m "..."
```

Se um repositório não tiver alterações na fase, não criar um commit artificial.

42. OBJETIVO FINAL

Ao final, o SentinelCore deverá demonstrar conhecimento em:

```
Python  
FastAPI  
Arquitetura  
Clean Architecture  
Arquitetura Hexagonal  
DDD  
PostgreSQL  
Redis  
RabbitMQ  
Docker  
Testes  
CI/CD  
Observabilidade  
Autenticação  
Autorização
```

Threat Modeling
Detecção de Ameaças
Incident Response
React
TypeScript
API Design
Sistemas Distribuídos

O projeto não deve parecer uma coleção de tecnologias.

Cada tecnologia deve possuir uma justificativa.

A pergunta sempre deve ser:

Qual problema essa tecnologia resolve?

43. OBJETIVO PROFISSIONAL

O SentinelCore deve ser um projeto que permita explicar:

- como o sistema funciona;
- por que os repositórios são separados;
- como o frontend conversa com o backend;
- como a API é versionada;
- como a autenticação funciona;
- como tokens são protegidos;
- como permissões são aplicadas;
- como eventos são registrados;
- como ameaças são detectadas;
- como riscos são calculados;
- como incidentes são criados;
- como respostas são executadas;
- como o sistema é observado;
- como o sistema pode escalar;
- quais decisões foram tomadas;
- quais alternativas foram rejeitadas;
- quais limitações existem.

O SentinelCore deve demonstrar que eu sei:

Pensar
↓
Projetar
↓
Separar responsabilidades
↓
Construir
↓

Integrar



Testar



Proteger



Observar



Operar



Evoluir

O projeto não deve ser apenas um CRUD.

Ele deve ser uma demonstração completa de engenharia de software e cibersegurança defensiva.